



**R.D. & S.H. NATIONAL COLLEGE & S.W.A. SCIENCE
COLLEGE**

Bandra, Mumbai-400050

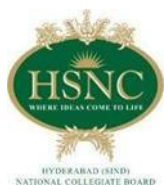
**Department of Computer Science &
Data Science**

Social Media Analytics

**M.Sc. Data Science Part I –Semester
II JOURNAL 2025-2026**

Course Code: - PSDSP516c

Seat No. : _____



R.D. & S.H. NATIONAL COLLEGE & S.W.A. SCIENCE COLLEGE

Bandra, Mumbai-400050.



Department of Computer Science & Data Science

CERTIFICATE

This is to certify that **Mr. Molla Zamir Hossain** of MSc. Part I (Sem II) class has satisfactorily completed **10/10** Practical in the subject of **Social Media Analytics** as a part of M.Sc. Degree Course in Data Science during the academic year 2025– 2026.

Faculty In-Charge

**Co-Ordinator
Department of Computer Science**

External Examiner

Index

Sr. No.	Title	Date	Page No.	Sign
1	Study Various • Social Media platforms (Facebook, twitter, YouTube etc) • Social Media analytics tools (Facebook insights, google analytics netlytic etc) • Social Media Analytics techniques and engagement metrics (page level, post level, member level) using Gephi Tool		4-5	
2	Scrape an online Social Media Site for Data. Use python to scrape information from twitter. Exploratory Data Analysis and visualization of Social Media Data.		6-7	
3	Create sociograms for the persons-by-persons network and the community-by- community network for a given relevant problem. Create a one-mode network and two-node network for the same. Datasets: les-Misérables, Airlines, Internet Core Routers.		8-9	
4	Develop Content (text, emoticons, image, audio, video) based social media analytics model for business. (e.g., Content Based Analysis: Topic, Issue, Trend, sentiment/opinion analysis, audio, video, image analytics)		10-11	
5	Develop Structure based social media analytics model for any business. (e.g., Structure Based Models - community detection, influence analysis)		12-13	
6	Develop a dashboard and reporting tool based on real time social media data Using Power BI		14-16	

7	Use Google Visualization Charts to analyse social media data		17-19	
8	Analyse social media data Network Analysis with Orange Software.		20	
9	Use Graph Neural Networks on the datasets (Planetoid Cora Dataset)/ Jazz Musicians Network.		21	
10	Analyse Twitter conversations to identify the most active and influential users using Machine Learning Algorithms with Gephi Tool.		22	

Practical 1

Aim:

Study Various

- **Social Media platforms (Facebook, twitter, YouTube etc)**
- **Social Media analytics tools (Facebook insights, google analytics netlytic etc)**
- **Social Media Analytics techniques and engagement metrics (page level, post level, member level) using Gephi Too**

1. Study of Various Social Media Platforms

Facebook

- One of the largest and most widely used social media platforms globally.
- Supports creation of personal profiles, business pages, groups, and events.
- Enables sharing of multimedia content including text, photos, videos, and live streams.
- Widely used for brand promotion, customer engagement, and community building.
- Offers targeted advertising and built-in analytics through Facebook Insights.

Twitter (now called X)

- A microblogging platform focused on real-time communication.
- Users post short messages called "tweets" (up to 280 characters).
- Supports likes, replies, retweets, and use of hashtags for categorization.
- Effective for tracking trends, news updates, and public discourse.
- Commonly used by influencers, media outlets, and public figures.

YouTube

- The largest video-sharing platform.
- Users can upload, view, comment on, and share videos.
- Offers features like subscriptions, monetization, playlists, and analytics.
- Used for entertainment, educational content, tutorials, marketing, and vlogging.
- Ideal for influencer collaborations and long-form video campaigns.

2. Study of Social Media Analytics Tools

Facebook Insights

- Built-in analytics tool for Facebook Pages.
- Tracks page-level metrics such as likes, reach, and demographics.
- Provides post-level performance data including likes, comments, and shares.
- Helps improve content strategy and audience targeting.

Google Analytics

- Web analytics platform used to track website traffic from social media.
- Measures referral traffic, bounce rate, session duration, and conversions.
- Offers insights into user behavior after clicking social media links.
- Supports integration with multiple social platforms for campaign tracking.

Netlytic

- Cloud-based social network and text analysis tool.
- Collects and analyzes public data from platforms like Twitter and YouTube.
- Performs keyword trend extraction, sentiment analysis, and interaction mapping.
- Ideal for communication research, digital marketing, and social science studies.

3. Study of Social Media Analytics Techniques and Engagement Metrics

Page-Level Metrics

- Measures the overall performance of a social media page.
- Common metrics include:
 - o Total likes or followers
 - o Reach (number of people who saw the page content)
 - o Impressions (number of times content was shown)
 - o Audience demographics (age, gender, location)
- Post-Level Metrics
 - Focuses on how individual posts perform.
 - Key metrics include:
 - o Likes, comments, shares (Facebook)
 - o Retweets, replies (Twitter)
 - o Views, likes, comments (YouTube)
 - o Engagement rate = $\text{Total interactions} \div \text{Total reach}$
- Helps identify effective content types and posting times.

Member-Level Metrics

- Analyzes user-level behavior and influence.
- Includes:
 - o Number of posts shared or created
 - o Interactions with other users (likes, replies, tags)
- Influence based on how often their content is engaged with. Useful for identifying top contributors, influencers, and community leaders.

4. Using Gephi for Social Media Analytics

What is Gephi?

Gephi is a powerful, open-source software platform used for exploring, analyzing, and visualizing networks and graphs. Think of it as Excel for network data, but visual and dynamic. It allows users to take data from social media (like Twitter retweets or Facebook interactions), transform it into a network graph, and then visually explore patterns, communities, and influential actors. Gephi is often described as the "Photoshop of network visualization" because of its highly customizable visual outputs and interactive features.

How Gephi Works (In a Social Media Context)

Step 1: Prepare Your Data

- Nodes: Represent users/accounts.
- Edges: Represent interactions (e.g., mentions, retweets, replies).
- You can collect data using tools like: Tweepy (Python) for Twitter data Netlytic or NodeXL for scraping and exporting social networks
- File formats Gephi supports: .gexf, .graphml, .csv, .net

Step 2: Load the Data into Gephi

- Import the dataset through Gephi's Data Laboratory.
- It automatically recognizes nodes and edges, and allows you to edit attributes (e.g., user names, tweet content, location).

Step 3: Run Network Metrics Gephi provides several built-in algorithms to measure the structure and influence within the network:

Use Case	Use Case
Influencer Detection	Identify the most central and influential users in a conversation (e.g., highest betweenness centrality).
Hashtag Analysis	Visualize how different hashtags are used together and by whom.
Community Detection	Discover clusters of users or groups discussing the same topic.
Misinformation Spread	Track how false information travels and identify origin points or key amplifiers.
Campaign Tracking	Visualize reach and engagement of a marketing campaign over a network.

Practical 2

Aim: Scrape an online Social Media Site for Data. Use python to scrape information from twitter. Exploratory Data Analysis and visualization of Social Media Data

Code:

```
import requests

from bs4 import BeautifulSoup

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from collections import Counter


quotes_data = []


for page in range(1, 6): # Scraping 5 pages
    url = f"http://quotes.toscrape.com/page/{page}/"
    response = requests.get(url)
    soup = BeautifulSoup(response.text, "html.parser")
    quotes = soup.find_all("div", class_="quote")
    for quote in quotes:
        text = quote.find("span", class_="text").text
        author = quote.find("small", class_="author").text
        tags = [tag.text for tag in quote.find_all("a", class_="tag")]
        quotes_data.append({
            "Quote": text,
            "Author": author,
            "Tags": tags
        })

df = pd.DataFrame(quotes_data)

print("\nFirst 5 rows of data:")
```

```

print(df.head())

print("\nTotal quotes scraped:", len(df))

df["Tags"] = df["Tags"].apply(lambda x: ", ".join(x))

print("\nMissing values:\n", df.isnull().sum())

author_counts = df["Author"].value_counts()

print("\nTop Authors:\n", author_counts.head())

all_tags = ", ".join(df["Tags"]).split(", ")

tag_counts = Counter(all_tags)

print("\nTop Tags:\n", tag_counts.most_common(10))

sns.set(style="whitegrid")

plt.figure(figsize=(8, 5))

top_authors = author_counts.head(5)

sns.barplot(x=top_authors.values, y=top_authors.index)

plt.title("Top Authors (Most Posts)")

plt.xlabel("Number of Quotes")

plt.ylabel("Author")

plt.show()

plt.figure(figsize=(8, 5))

top_tags = dict(tag_counts.most_common(5))

sns.barplot(x=list(top_tags.values()), y=list(top_tags.keys()))

plt.title("Trending Tags")

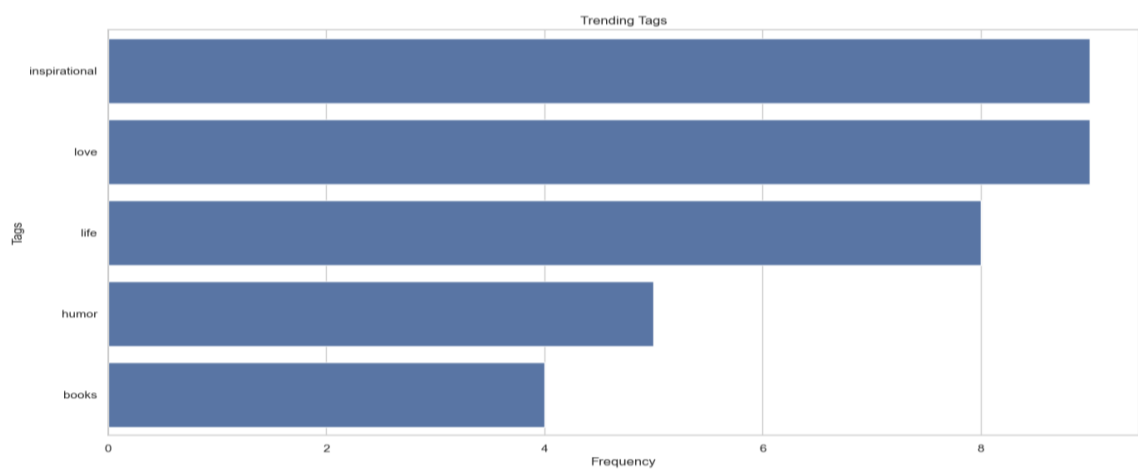
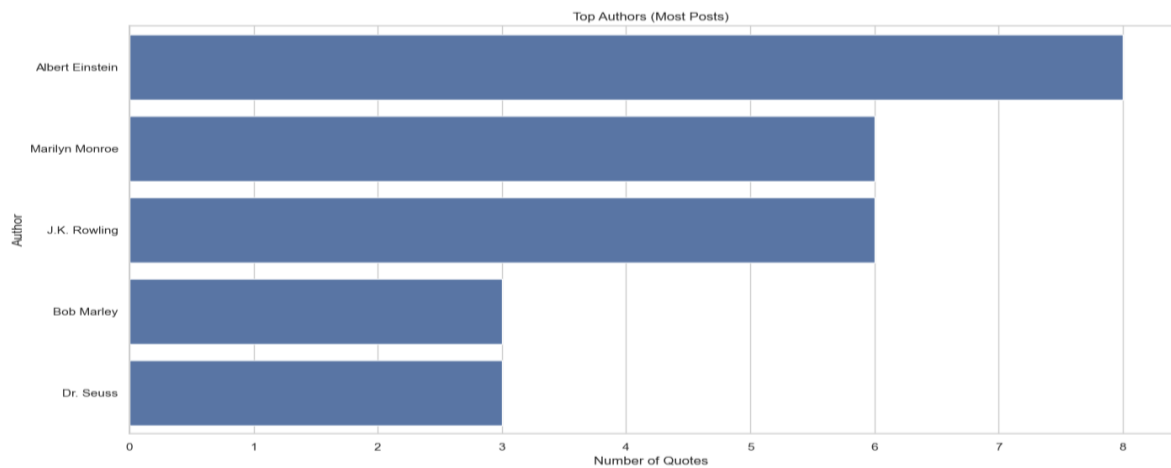
plt.xlabel("Frequency")

plt.ylabel("Tags")

plt.show()

```

Output:



```
First 5 rows of data:
      Quote ...
0 "The world as we have created it is a process ... [change, deep-thoughts, thinking, world]
1 "It is our choices, Harry, that show what we t... [abilities, choices]
2 "There are only two ways to live your life. On... [inspirational, life, live, miracle, miracles]
3 "The person, be it gentleman or lady, who has ... [aliteracy, books, classic, humor]
4 "Imperfection is beauty, madness is genius and... [be-yourself, inspirational]

[5 rows x 3 columns]

Total quotes scraped: 50

Missing values:
Quote      0
Author      0
Tags        0
dtype: int64

Top Authors:
Author
Albert Einstein      8
Marilyn Monroe       6
J.K. Rowling         6
Bob Marley           3
Dr. Seuss            3
Name: count, dtype: int64

Top Tags:
[('inspirational', 9), ('love', 9), ('life', 8), ('humor', 5), ('books', 4), ('reading', 4), ('friends', 3), ('friendship', 3), ('paraphrased', 2), ('simile', 2)]
```

Practical 3

Aim: Create sociograms for the persons-by-persons network and the community-by-community network for a given relevant problem. Create a one-mode network and two-node network for the same. Datasets: les-Misérables, Airlines, Internet Core Routers.

Code:

```
import networkx as nx

import matplotlib.pyplot as plt

G = nx.les_miserables_graph()

print("Total Nodes:", G.number_of_nodes())

print("Total Edges:", G.number_of_edges())


plt.figure(figsize=(8,6))


pos = nx.spring_layout(G, seed=42)

nx.draw(G, pos, with_labels=True, node_size=300, font_size=8)


plt.title("One-Mode Network (Person-Person)")

plt.show()


from networkx.algorithms import community

communities = list(community.greedy_modularity_communities(G))

print("\nNumber of Communities:", len(communities))


C_graph = nx.Graph()


for i in range(len(communities)):

    C_graph.add_node(i)


# Add edges if communities share connections

for i in range(len(communities)):
```

```

for j in range(i+1, len(communities)):
    for node1 in communities[i]:
        for node2 in communities[j]:
            if G.has_edge(node1, node2):
                C_graph.add_edge(i, j)
                break

```

```

plt.figure(figsize=(6,5))
pos = nx.spring_layout(C_graph, seed=42)
nx.draw(C_graph, pos, with_labels=True, node_color='orange')

```

```

plt.title("Community-by-Community Network")
plt.show()

```

```

B = nx.Graph()

```

```

for i, comm in enumerate(communities):
    B.add_node(f'Community_{i}', bipartite=0)
    for person in comm:
        B.add_node(person, bipartite=1)
        B.add_edge(person, f'Community_{i}')

```

```

plt.figure(figsize=(8,6))
pos = nx.spring_layout(B, seed=42)
nx.draw(B, pos, with_labels=False, node_size=200)

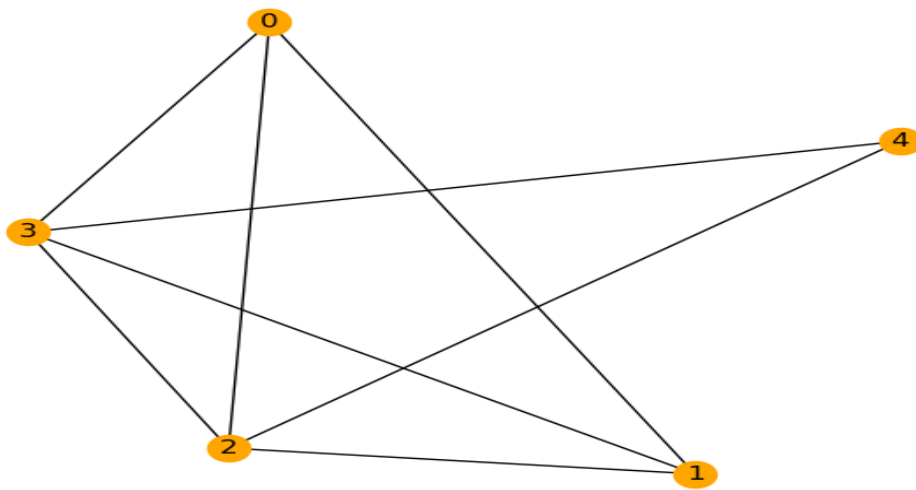
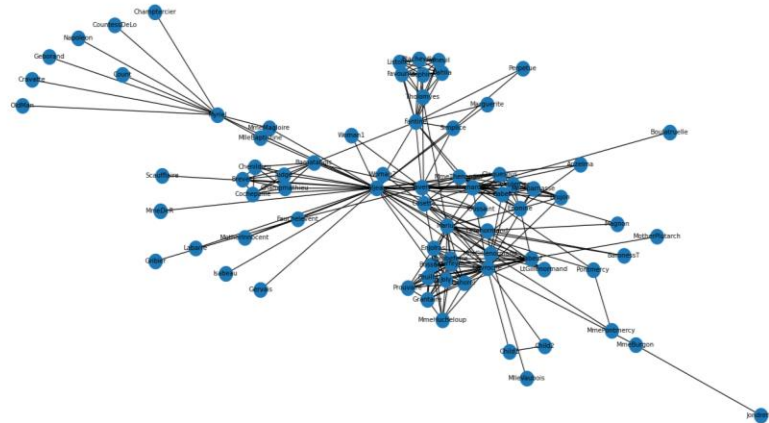
```

```

plt.title("Two-Mode Network (Person-Community)")
plt.show()

```

Output:



Practical 4

Aim: Develop Content (text, emoticons, image, audio, video) based social media analytics model for business. (e.g., Content Based Analysis: Topic, Issue, Trend, sentiment/opinion analysis, audio, video, image analytics)

Code:

```
import pandas as pd

from textblob import TextBlob

from collections import Counter

data = {
    "Post": [
        "I love this product 😊 #quality",
        "Worst service ever 😞 #bad",
        "Amazing experience 😍 #happy",
        "Not good, very slow 😞 #poor",
        "Great value for money 🙌 #deal"
    ],
    "Image_Label": ["product", "service", "experience", "service", "product"],
    "Audio_Label": ["positive tone", "angry tone", "happy tone", "sad tone", "positive tone"],
    "Video_Label": ["review", "complaint", "review", "complaint", "review"]
}

df = pd.DataFrame(data)

print("\nData:\n", df)

def get_sentiment(text):
    score = TextBlob(text).sentiment.polarity
    if score > 0:
        return "Positive"
```

```
elif score < 0:
    return "Negative"
else:
    return "Neutral"
```

```
df["Sentiment"] = df["Post"].apply(get_sentiment)
```

```
print("\nSentiment:\n", df[["Post", "Sentiment"]])
```

```
emojis = ["😄", "😞", "😍", "😓", "👍"]
```

```
emoji_count = {}
```

```
for e in emojis:
    emoji_count[e] = df["Post"].str.count(e).sum()
```

```
print("\nEmoji Count:\n", emoji_count)
```

```
hashtags = []
```

```
for post in df["Post"]:
    words = post.split()
    for w in words:
        if w.startswith("#"):
            hashtags.append(w)
```

```
tag_count = Counter(hashtags)
```

```
print("\nTrending Topics:\n", tag_count)
```

```

image_counts = df["Image_Label"].value_counts()

print("\nImage Content:\n", image_counts)

audio_counts = df["Audio_Label"].value_counts()

print("\nAudio Tone:\n", audio_counts)

video_counts = df["Video_Label"].value_counts()

print("\nVideo Type:\n", video_counts)

print("\n--- BUSINESS INSIGHTS ---")

print("Most common sentiment:", df["Sentiment"].value_counts().idxmax())

print("Top hashtag:", tag_count.most_common(1))

print("Top image type:", image_counts.idxmax())

print("Top video type:", video_counts.idxmax())

```

Output:

```

Data:
   Post Image_Label Audio_Label Video_Label
0  I love this product 😊 #quality product positive tone review
1  Worst service ever 😡 #bad service angry tone complaint
2  Amazing experience 😄 #happy experience happy tone review
3  Not good, very slow 😞 #poor service sad tone complaint
4  Great value for money 🤝 #deal product positive tone review

Sentiment:
   Post Sentiment
0  I love this product 😊 #quality Positive
1  Worst service ever 😡 #bad Negative
2  Amazing experience 😄 #happy Positive
3  Not good, very slow 😞 #poor Negative
4  Great value for money 🤝 #deal Positive

Emoji Count:
{'😊': 1, '😡': 1, '😄': 1, '😞': 1, '🤝': 1}

Trending Topics:
Counter({'#quality': 1, '#bad': 1, '#happy': 1, '#poor': 1, '#deal': 1})

Image Content:
Image_Label
product      2
service      2
experience    1
Name: count, dtype: int64

Audio Tone:
Audio_Label
positive tone      2
angry tone         1
happy tone         1
sad tone           1
Name: count, dtype: int64

Video Type:
Video_Label
review      3
complaint   2
Name: count, dtype: int64

--- BUSINESS INSIGHTS ---
Most common sentiment: Positive
Top hashtag: [('quality', 1)]
Top image type: product
Top video type: review

```

Practical 5

Aim: Develop Structure based social media analytics model for any business. (e.g., Structure Based Models -community detection, influence analysis)

Code:

```
import networkx as nx
import matplotlib.pyplot as plt

# Create a graph
G = nx.Graph()

# Add users (nodes)
users = ["A", "B", "C", "D", "E", "F", "G", "H"]
G.add_nodes_from(users)

# Add connections between users (edges)
connections = [
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D"),
    ("D", "E"),
    ("E", "F"),
    ("F", "G"),
    ("G", "H"),
    ("E", "H")
]

G.add_edges_from(connections)

from networkx.algorithms.community import greedy_modularity_communities

communities = greedy_modularity_communities(G)

print("Communities Detected:")
for i, community in enumerate(communities):
    print(f"Community {i+1}: {list(community)}")

print("\nInfluence Analysis using Degree Centrality:")

centrality = nx.degree_centrality(G)

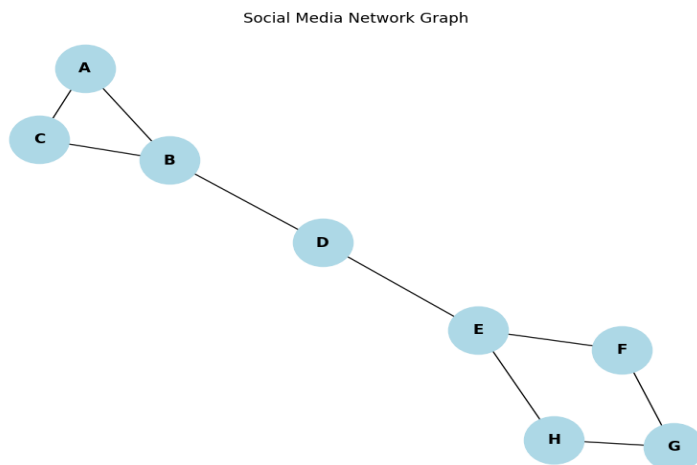
for user, score in centrality.items():
    print(user, "=", round(score, 2))

# Find most influential user
most_influential = max(centrality, key=centrality.get)
```

```
print("\nMost Influential User:", most_influential)
```

```
plt.figure(figsize=(8,6))  
pos = nx.spring_layout(G)  
nx.draw(  
    G,  
    pos,  
    with_labels=True,  
    node_color='lightblue',  
    node_size=2000,  
    font_size=12,  
    font_weight='bold'  
)  
plt.title("Social Media Network Graph")  
plt.show()
```

Output:



Practical 6

Aim: Develop a dashboard and reporting tool based on real time social media data Using Power BI

Create Charts and Reports in Power BI

1. What is a Chart or Report

In Power BI, a Chart is a visual representation of data such as bar, line, or pie charts. A Report is a collection of visuals that helps analyze and understand data in one place. Power BI allows users to create interactive and attractive reports easily.

2. Sample Data

Create the following dataset in Excel and save as **SalesReport.xlsx**:

	Product Category	Month	Sales	Profit
A	Electronics	Jan	5000	1200
B	Electronics	Jan	4000	1000
C	Furniture	Feb	3000	800
D	Furniture	Feb	3500	900
E	Clothing	Mar	4500	1100
F	Clothing	Mar	5000	1300

Import Steps:

Home → Get Data → Excel → Select file → Load

3. Create Visuals (Charts)

a. Bar Chart (Sales by Category)

- Select *Clustered Column Chart*
- Axis → Category
- Values → Sales

b. Pie Chart (Profit by Product)

- Select *Pie Chart*
- Legend → Product
- Values → Profit

c. Line Chart (Sales over Months)

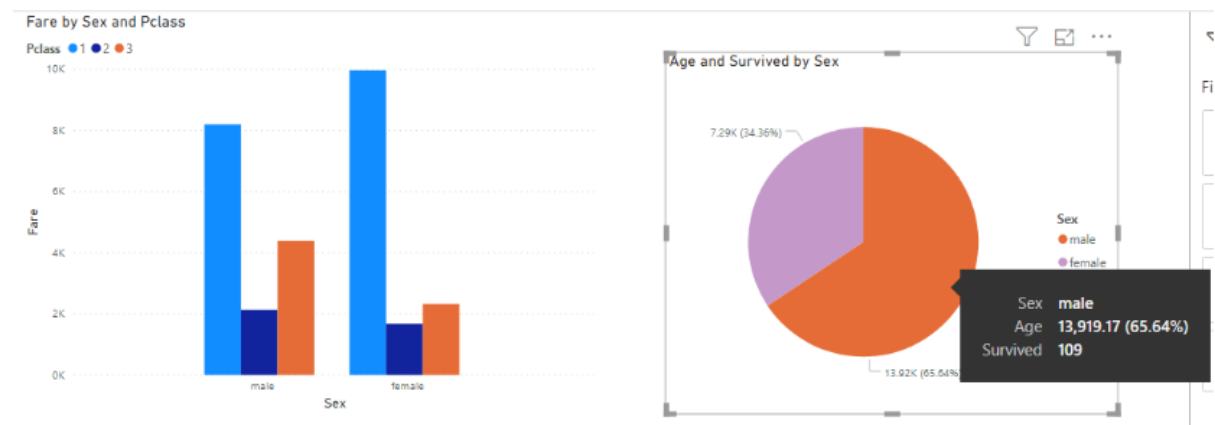
- Select *Line Chart*
- X-axis → Month
- Values → Sales
-

4. Report Page Layout

Arrange visuals neatly:

- Bar chart → Left
- Pie chart → Right
- Line chart → Bottom

Resize by dragging edges.



Practical 7

Aim: Use Google Visualization Charts to analyze social media data

Code:

```
<!DOCTYPE html>
<html>
<head>
<title>Social Media Analytics</title>

<!-- Load Google Charts -->
<script type="text/javascript" src="https://www.gstatic.com/charts/loader.js"></script>

<script type="text/javascript">

// Load chart package
google.charts.load('current', {'packages':['corechart']});

// Draw chart after loading
google.charts.setOnLoadCallback(drawChart);

function drawChart() {

// Social media data
var data = google.visualization.arrayToDataTable([
['Platform', 'Likes', 'Comments', 'Shares'],
['Instagram', 500, 200, 150],
['Facebook', 400, 180, 120],
['Twitter', 300, 100, 80],
['YouTube', 700, 350, 250]
]);

// Chart options
var options = {
title: 'Social Media Analytics',
width: 800,
height: 500,
legend: { position: 'top' }
};

// Create column chart
var chart = new google.visualization.ColumnChart(document.getElementById('chart_div'));

// Draw chart
chart.draw(data, options);
}

</script>
```

```
</head>

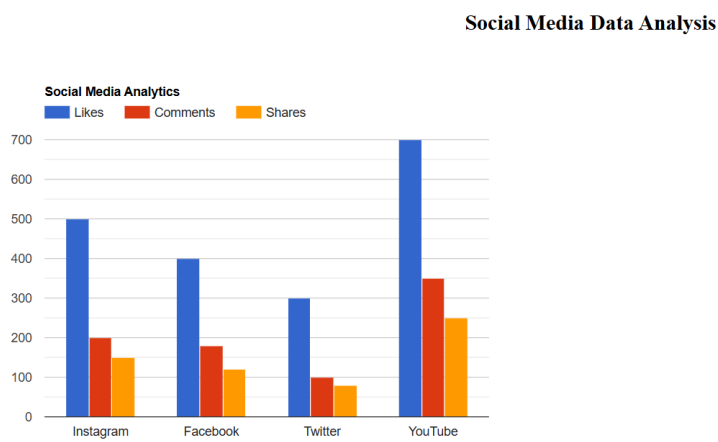
<body>

<h2 align="center">Social Media Data Analysis</h2>

<!-- Chart container -->
<div id="chart_div"></div>

</body>
</html>
```

Output:



Practical 8

Aim: Analyze social media data Network Analysis with Orange Software

Code:

```
import networkx as nx
import matplotlib.pyplot as plt

# Step 1: Create sample social media graph
G = nx.Graph()
edges = [("UserA", "UserB"), ("UserA", "UserC"), ("UserB", "UserC"), ("UserC", "UserD"),
("UserD", "UserE"), ("UserE", "UserF"), ("UserF", "UserG"), ("UserG", "UserH"), ("UserH",
"UserA")]
G.add_edges_from(edges)

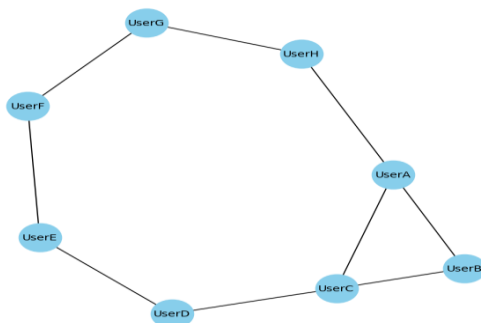
# Step 2: Community detection (using greedy modularity)
from networkx.algorithms.community import greedy_modularity_communities
communities = list(greedy_modularity_communities(G))
print("Communities:", communities)

# Step 3: Influence analysis (centrality measures)
degree centrality = nx.degree_centrality(G)
betweenness centrality = nx.betweenness_centrality(G)
pagerank = nx.pagerank(G)
print("Degree Centrality:", degree_centrality)
print("Betweenness Centrality:", betweenness_centrality)
print("PageRank:", pagerank)

# Step 4: Visualization
plt.figure(figsize=(6,6))
nx.draw(G, with_labels=True, node_color="skyblue", node_size=1200, font_size=10)
plt.show()
```

Output:

```
Communities: [frozenset({'UserA', 'UserC', 'UserB'}), frozenset({'UserH', 'UserF', 'UserG'}), frozenset({'UserE',
'UserD'})]
Degree Centrality: {'UserA': 0.42857142857142855, 'UserB': 0.2857142857142857, 'UserC':
0.42857142857142855, 'UserD': 0.2857142857142857, 'UserE': 0.2857142857142857, 'UserF':
0.2857142857142857, 'UserG': 0.2857142857142857, 'UserH': 0.2857142857142857}
'UserE': 0.1169973090700547, 'UserF': 0.11819873113296023, 'UserG': 0.1169973090700547, 'UserH':
0.1129735734361737}
```



Practical 9

Aim: Use Graph Neural Networks on the datasets (Planetoid Cora Dataset)/ Jazz Musicians Network.

Code:

```
# Import libraries
import torch
import torch.nn.functional as F

from torch_geometric.datasets import Planetoid
from torch_geometric.nn import GCNConv

# Load Cora Dataset
dataset = Planetoid(root='data/Cora', name='Cora')

data = dataset[0]

print("Dataset:", dataset)
print("Number of Nodes:", data.num_nodes)
print("Number of Edges:", data.num_edges)

class GCN(torch.nn.Module):

    def __init__(self):
        super().__init__()

        self.conv1 = GCNConv(dataset.num_node_features, 16)
        self.conv2 = GCNConv(16, dataset.num_classes)

    def forward(self, data):

        x, edge_index = data.x, data.edge_index

        # First GCN Layer
        x = self.conv1(x, edge_index)
        x = F.relu(x)

        # Second GCN Layer
        x = self.conv2(x, edge_index)

        return F.log_softmax(x, dim=1)

# Create model
model = GCN()

# Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

```

model.train()

for epoch in range(100):

    optimizer.zero_grad()

    out = model(data)

    loss = F.nll_loss(out[data.train_mask], data.y[data.train_mask])

    loss.backward()

    optimizer.step()

    if epoch % 10 == 0:
        print("Epoch:", epoch, "Loss:", loss.item())

model.eval()

pred = model(data).argmax(dim=1)

correct = (pred[data.test_mask] == data.y[data.test_mask]).sum()

accuracy = int(correct) / int(data.test_mask.sum())

print("\nAccuracy:", round(accuracy * 100, 2), "%")

```

Output:

```

... Downloading https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.x
Downloading https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.tx
Downloading https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.allx
Downloading https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.y
Downloading https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.ty
Downloading https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.ally
Downloading https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.graph
Downloading https://github.com/kimiyoung/planetoid/raw/master/data/ind.cora.test.index
Processing...
Done!
Dataset: Cora()
Number of Nodes: 2708
Number of Edges: 10556
Epoch: 0 Loss: 1.953129529953003
Epoch: 10 Loss: 0.6688180565834045
Epoch: 20 Loss: 0.1227484717965126
Epoch: 30 Loss: 0.02620449848473072
Epoch: 40 Loss: 0.009512525983154774
Epoch: 50 Loss: 0.005243998020887375
Epoch: 60 Loss: 0.0037055998109281063
Epoch: 70 Loss: 0.002965832594782114
Epoch: 80 Loss: 0.002522306516766548
Epoch: 90 Loss: 0.0022105516400188208
\nAccuracy: 77.5 %

```

Practical 10

Aim: Analyze Twitter conversations to identify the most active and influential users using Machine Learning Algorithms with Gephi Tool.

Code:

```
# Import libraries
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

data = {
    "User": [
        "A", "A", "B", "C", "D",
        "E", "F", "G", "H", "B"
    ],
    "Reply_To": [
        "B", "C", "C", "D", "E",
        "F", "G", "H", "A", "E"
    ]
}

df = pd.DataFrame(data)

print("Twitter Conversation Data:")
print(df)

G = nx.from_pandas_edgelist(
    df,
    source='User',
    target='Reply_To'
)

print("\nMost Active Users:")

activity = df['User'].value_counts()

print(activity)

print("\nInfluential Users using Degree Centrality:")

centrality = nx.degree_centrality(G)

for user, score in centrality.items():
    print(user, "=", round(score, 2))
```

```

# Most influential user
most_influential = max(centrality, key=centrality.get)

print("\nMost Influential User:", most_influential)

print("\nNormalized Influence Scores:")

scores = list(centrality.values())

scaler = MinMaxScaler()

normalized = scaler.fit_transform(
    [[s] for s in scores]
)

for user, score in zip(centrality.keys(), normalized):
    print(user, "=", round(score[0], 2))

plt.figure(figsize=(8,6))

pos = nx.spring_layout(G)

nx.draw(
    G,
    pos,
    with_labels=True,
    node_color='skyblue',
    node_size=2500,
    font_size=12,
    font_weight='bold'
)

plt.title("Twitter Conversation Network")
plt.show()

```

Output

```

... Twitter Conversation Data:
  User Reply_To
0      A      B
1      A      C
2      B      C
3      C      D
4      D      E
5      E      F
6      F      G
7      G      H
8      H      A
9      B      E

Most Active Users:
User
A    2
B    2
C    1
D    1
E    1
F    1
G    1
H    1
Name: count, dtype: int64

... Influential Users using Degree Centrality:
A = 0.43
B = 0.43
C = 0.43
D = 0.29
E = 0.43
F = 0.29
G = 0.29
H = 0.29

Most Influential User: A

Normalized Influence Scores:
A = 1.0
B = 1.0
C = 1.0
D = 0.0
E = 1.0
F = 0.0
G = 0.0
H = 0.0

```

